MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión E del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_pineda_giovanni
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/pineda_giovanni/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código

¿Qué significa que un objeto sea una instancia de una clase? Explica si dos instancias deben tener iguales valores en sus atributos.

Tu respuesta

```
significa que el objeto sera donde trabajremos fuera de la clase y se puede ocupar dentro de
la clase si deberian tener valores iguales en sus atributos ya que si no, no podria compilar
el codigo ya que tecnicamente son lo mismo
```

Corrección. La respuesta no explica que una instancia es un objeto concreto creado a partir de la clase y afirma que dos instancias deben tener valores iguales para compilar, lo cual es falso. Lo correcto es que comparten la definición de atributos y métodos, pero sus atributos de instancia pueden tener valores distintos.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué diferencia hay entre un parámetro y un argumento de un método? Ilústrala con una llamada sencilla.

Tu respuesta

```
el parametro delimita los contenidos de un objeto por decirlo de alguna manera mientras que el
argumento de un metodo es el valor o funcion que hace que el metodo funcione
```

Corrección. Reconoce parcialmente que el argumento es un valor que se le da al método, pero describe mal el parámetro y no da ejemplo. El parámetro es la variable declarada en la cabecera (int cantidad en cargar(int cantidad)); el argumento es el valor pasado en la llamada (el 3 en cargar(3)).

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué es el ámbito de una variable local? Da un ejemplo de dónde se puede usar y dónde deja de estar disponible.

Tu respuesta

```
una variable local es lo que unicamente se puede ejecutar dentro del metodo y no en todo constructor
```

Corrección. Conserva la idea válida de que la variable local solo se usa dentro de donde se declaró, pero la mezcla con 'ejecutar' y 'constructor' y no da un ejemplo de límite de bloque. El ámbito es la región donde el nombre es visible: una variable declarada dentro de las llaves de un if solo existe dentro de ese bloque.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué guarda una variable de tipo arreglo? Explica qué ocurre al asignarla a otra variable del mismo tipo.

Tu respuesta

```
una variable de tipo arreglo guarda funciones al asignarle a otra el mismo tipo el codigo no deberia de funcionar ya que se confundira con ambos arreglos
```

Corrección. Es incorrecto que un arreglo guarde funciones y que la asignación no funcione. La variable guarda una referencia al arreglo; al asignarla a otra se copia la referencia y ambas apuntan al mismo arreglo (alias), por lo que un cambio por una se ve por la otra.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Qué hace break dentro de un case de un switch clásico? Explica qué puede ocurrir si falta.

Tu respuesta

```
un break es la salida dentro del switch si el break falta puede volverse un bulce donde nunca va a terminar o no habra cambios al ejecutarlo.
```

Corrección. Acierta en que break es la salida del switch, pero la consecuencia es incorrecta: sin break no se forma un ciclo infinito. Si falta, la ejecución continúa por los case siguientes (fall-through) sin volver a comparar sus etiquetas.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer original = 7;
Integer copia = original;
int vuelta = 0;
do {
    original = original + 2;
    System.out.println(original / 2);
    vuelta++;
} while (vuelta < 3);
String texto = "E" + copia + 1;
System.out.println(texto);
System.out.println((double) (original / 2));
```

Tu respuesta

```
4 cuando vueta sea mas de 3
E8 mientras vuelta se mantenga menor que 3
4.5 mientras vuelta se mantenga menor que 3
```

Corrección. La salida correcta es 4 / 5 / 6 / E71 / 6.0. El primer 4 es correcto y 'E8' reconoce el prefijo 'E' y que copia vale 7, pero sumó 7+1 en lugar de concatenar ("E"+7+1 da E71); faltan 5 y 6 del ciclo do-while y 4.5 ignora que 13/2 es división entera antes del cast, que da 6.0.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = null;
boolean igual = codigo.equals("R3");
System.out.println(igual);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
creo que compila pero truena ya que al ejecutarlo el nulo no se puede comparar con R3
```

Corrección. Clasificación correcta: compila pero truena. La causa se acerca pero es imprecisa: el problema no es 'comparar con R3', sino invocar el método equals sobre una referencia null, lo que lanza NullPointerException.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double voltaje = 7.6;
int lectura = voltaje;
System.out.println(lectura);
```

Tu respuesta

Clasificación

Funciona

Causa o salida

funciona pero al tener el lectura como un int no imprimira el 7.6 si no unicamente 7

Corrección. No funciona: no compila, porque Java no convierte implícitamente un double a int (habría que escribir (int) voltaje). Suponer que trunca a 7 solo ocurriría con un cast explícito.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer reserva = 4;
int nueva = reserva + 3;
System.out.println(nueva);
```

Tu respuesta

Clasificación

Compila pero truena

Causa o salida

compilaria pero no se podria ejecutar ya que primero deberiamos hacer un unboxing del integer

Corrección. Funciona e imprime 7: Java hace el unboxing automático del Integer para sumar, así que no hace falta desempacarlo a mano ni truena. La clasificación y la causa son incorrectas.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada banda nace vacía con capacidad propia fija. El límite de seguridad común es 12 piezas. agregarPieza() suma una y consultarPiezas() responde cuántas hay. Supón que solo se llama agregarPieza cuando ambos límites lo permiten.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cuál es final de instancia y cuál static final: capacidad o límite de seguridad?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
scanner....
{
int piezas = 0;

switch {

1. " agregar piezas"
while (piezas < 12);
piezas++;
else (piezas > 12)
System.out.println("haz llegado a las piezas maximas" + piezas);

2 "consultar piezas"
System.out.println ("hay" + piezas + "en total")
    }
}
```

Justificaciones

```
primero declaro mi entero piezas que se inicie en 0 ya que pide una banda vacia
el switch es para poder elegir si meterle una pieza mas o ver cuantas hay
while es para ponerle el tope de seguridad de 12 piezas
```

Corrección. No escribió una clase: usó un switch, un while y un println, cuando se pedían atributos, constructor y métodos de 1 a 3 líneas sin estructuras de control. Van bien el contador int piezas que inicia en 0, con su justificación, y la idea de agregar con piezas++ y de consultar el total. Faltan capacidad como final de instancia, LIMITE_SEGURIDAD = 12 como static final, el constructor con this y la respuesta al reto (capacidad final de instancia, límite static final).

R4 · Código con repetición

Pregunta

Un equipo parte con carga unidades. En cada paso suma aporte. Devuelve cuántos pasos necesita para alcanzar o superar meta usando while. Si ya llegó, devuelve 0. Carga y meta están entre 0 y 10000; aporte está entre 1 y 100. No uses una fórmula de división.

```
static int pasosCarga(int carga, int aporte, int meta)
```

Ejemplos (argumentos → retorno):

5, 4, 16 → 3 · 16, 4, 16 → 0 · 20, 4, 16 → 0

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
// comprendi que pedia que un equipo de senderistas ocupan llegar a una meta y cada que
caminan cada uno aporta al contador tanto la meta como la carga las escogi yo ya que pedia que
no leyeramos el teclado
static int pasosCarga(int carga, int aporte, int meta)
int aporte = 20
int meta = 5,000
while (carga < meta)
int carga = carga + aporte
//me parece que le falta aqui para actualizar carga + aporte = carga para que ocupe de nuevo
la suma y siga aumentando el numero pero la verdad no recuerdo como se hace
System.out.println(carga)
else (carga = meta)
system.out.println "0"
```

Corrección. Cabecera correcta, usa while con la condición carga < meta, que también se detiene al sobrepasar, y acumula aporte en carga, aunque redeclara y fija aporte y meta en lugar de usar los parámetros. Falta un contador de pasos (int pasos = 0; pasos++ dentro del ciclo) y return pasos; imprime en vez de retornar, el else después del while no existe en Java y faltan llaves y punto y coma.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Decide si un banco de pruebas puede arrancar. Recibes el porcentaje de batería y si la tapa está cerrada. Aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else, con && y ||. No imprimas ni leas del teclado.

```
static String permiso(int bateria, boolean tapaCerrada)
```

Regla o valor recibido	Texto devuelto
Batería menor que 0 o mayor que 100	"INVALIDA"
Tapa cerrada y batería al menos 60	"NORMAL"
Tapa cerrada y batería al menos 20	"AHORRO"
Cualquier otro caso	"BLOQUEADO"

Tu respuesta

```
static String permiso(int bateria, boolean tapaCerrada)

if { (bateria < 0)|| (bateria > 100)
System.out.println ("Invalida")
else {
```

```
(bateria < 60) && tapaCerrada
System.out.println "NORMAL"
}
else{ (bateria < 20) && tapaCerrada
System.out.println "AHORRO"
}
else { tapaAbierta
System.out.println "Bloqueado"
    }
}
```

Corrección. Cabecera correcta, la validación bateria < 0 || bateria > 100 es correcta, el orden de reglas es el debido y el caso final bloquea con la tapa abierta. Las comparaciones están invertidas: debía ser tapaCerrada && bateria >= 60 para NORMAL y tapaCerrada && bateria >= 20 para AHORRO, en una cadena if / else if / else. Además imprime en lugar de hacer return de "INVALIDA", "NORMAL", "AHORRO" o "BLOQUEADO", y la sintaxis del if y de los println no es válida.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.